Test Report

STAR: Spatial Topography Accessibility Robot

Unit, Integration, and Hardware-in-the-Loop Test Results

Team Locked Inc

Cesar Magana	Software (firmware, gateway, ROS2, UI, compliance, infra)
Brighton Sikarskie	Embedded Development and Schematic Design
Jeremie Hockey	PCB Design and Layout
Michael Norton	Mechanical Design
Shawn Ciaciura	Schematic Review
Jared Bartz	Schematic Review

Course

ESET 420 – Senior Design II – Dr. Lee Hudson

Institution

Texas A&M University
Department of Electronic Systems Engineering Technology (ESET)
ESET Senior Capstone, Spring 2026

Advisors and Sponsors

Dr. Logan Porter – Sponsor and Technical Advisor

April 28, 2026

Version 1.0 – Capstone Submission

Contents

List of Figures

List of Tables

1 Introduction

1.1 Purpose

This report documents the verification and validation effort for the STAR software. It enumerates every automated test suite, identifies the subsystem each suite exercises, reports pass/fail status and measured coverage where available, and records the hardware bring-up tests performed against real silicon. Its structure is informed by IEEE 29119-3 (Test Documentation) and IEEE 829 legacy practice, adapted to the multi-language, multi-target STAR codebase.

1.2 Scope

The report covers eight software subsystems: the RX72N real-time firmware, the Go gateway, the ROS2 Jazzy autonomy stack, the TypeScript operator UI, the Python compliance engine, the Protocol Buffers schema repository, the virtual-RX72N simulator, and the CI/CD automation that drives the whole pipeline. Physical PCB and mechanical verification are out of scope for this report.

1.3 Companion Documents

Architecture and design details are described in the STAR Software Description Document (SDD), and operational procedures in the STAR System and Software User Manual (SSUM). Where this report refers to a design decision without elaboration, the SDD is authoritative.

1.4 Test Strategy at a Glance

STAR applies a four-tier strategy:

1. **Unit tests** run on host with native compilers and mock-injected hardware (Unity for firmware, Go's `testing` for the gateway, `pytest` for compliance, Vitest for UI and Protobuf TypeScript, C Unity for nanopb).

2. **Integration tests** exercise multi-component flows via an in-process simulator (`virtual_rx72n`) and ROS2 launch tests.

3. **Hardware-in-the-loop (HIL) tests** execute nightly on a dedicated fixture and against bench firmware-bring-up dirs.

4. **CI-enforced quality gates** block merges on coverage regression, lint failure, protobuf breaking changes, and 7-bit ASCII violations.

2 Test Environment

2.1 Host Toolchain

Unit-test runs target x86_64 Linux (Ubuntu 24.04) on CI and on developer workstations. Toolchains include GNU Make, CMake, the GNURX cross-compiler for firmware host-test fallback, Go 1.26, Node 22 with Vitest, Python 3.12, and ROS2 Jazzy.

2.2 Target Hardware

HIL and bench tests run against: Renesas RX72N (4 MB Flash, 512 KB SRAM) on the STAR custom PCB, four DRV8263H H-bridge motor drivers, quadrature encoders (341 PPR), Bosch BNO055 IMU, BMP280 barometer, HC-SR04 ultrasonic sensors, RPLiDAR C1 2D LiDAR, and a Raspberry Pi 5 host.

2.3 Test Frameworks

Table 1 maps each subsystem to its test framework.

Table 1: Test frameworks in use by subsystem.

Subsystem	Framework	Notes
star-rx72n-firmware	Unity v2.6.1	CMake FetchContent; 37 mocks incl. ThreadX
star-gateway	Go `testing`	With `-race` and `-coverprofile`
star-ros2 (C++)	ament_cmake_gtest	colcon test integration
star-ros2 (Python)	ament_python / pytest	launch_test for simulation
star-ui	Vitest	jsdom environment, testing-library
compliance-engine	pytest	Coverage via `pytest-cov`
star-proto (Go)	Go `testing`	Serialization round-trips
star-proto (TS)	Vitest	Serialization + conformance
star-proto (nanopb)	Unity	C serialization tests

3 Firmware Unit Tests (RX72N)

3.1 Scope and Inventory

Unity-based tests live under `star-rx72n-firmware/tests/`. Sixty test source files exercise the firmware libraries in isolation, using function-pointer dependency injection to replace hardware with mocks. Tests are grouped by concern: cores (`rx_crc8`, `rx_crc32`, `rx_frame`, `rx_fec`, `rx_harq`, `rx_pid`, `rx_error_handler`, `rx_pin_validator`, `rx_register_guard`); hardware abstraction (`gpio_hal`, `uart_hal`, `rx_gptw_staggered`, `rx_mpc`, `rx_usb`, `rx_usb_multiport`, `rx_usb_comm`); communication (`rx_comm_manager`, `rx_spi_comm`, `rx_bus_manager`, `rx_bus_i2c`, `rx_bus_gpio`, `rx_bus_onewire`, `rx_bus_adc`, `rx_nanopb`); sensors (`rx_hcsr04`, `rx_obstacle_detect`, `rx_ds18b20`, `rx_encoder`, `rx_encoder_tpu`, `rx_bmp280`, `rx_bno055`); motor control (`rx_motor`); and the seven ThreadX task entry points.

3.2 Results

Table 2 summarizes the most recent CI-captured JUnit artifact at `e2-studio-star-rx72n-firmware/tests/build`

Table 2: Firmware unit test results (JUnit XML, 2026-03-01).

Metric	Value
Test suites executed	46
Test cases executed	46
Failures	0
Errors	0
Skipped	0
Overall status	**PASS**
Wall-clock duration (per case)	15 ms to 25 ms

3.3 Coverage Gate

The firmware CI workflow `.github/workflows/firmware-unit-tests.yml` generates an lcov HTML coverage report and applies a strict gate: **100 percent line, branch, and function coverage on**

`libs/` (generated code and ThreadX task bodies are excluded). A regression below this threshold blocks merge.

3.4 Notable Test Categories

- **PID correctness** (`test_rx_pid`): checks the discrete velocity-form update against a scalar reference implementation and verifies anti-windup clamping behavior.
- **CRC-32 conformance** (`test_rx_crc32`): agrees with the Go gateway's CRC-32 on a shared fixture vector, guaranteeing cross-language byte identity.
- **Frame go-compat** (`test_frame_go_compat`): encodes a set of payloads in firmware code and checks that the Go frame decoder accepts them bit-exactly.
- **FEC and HARQ** (`test_rx_fec`, `test_rx_harq`): validates the rate-1/2 K=7 convolutional encoder against the Viterbi decoder and verifies Chase Combining soft-bit accumulation across simulated retransmissions.

3.5 Hardware Bring-Up Tests

In addition to unit tests, stand-alone firmware trees at the root of `star-rx72n-firmware/` provide on-target bring-up programs used during PCB validation. Table 3 lists each test and its current status as exercised on the bench.

Table 3: Hardware bring-up test directories.

Directory	What it verifies	Status
`motor_spin_test/`	Open-loop PWM drive on all four motors	**PASS**
`pwm_test/`	20 kHz PWM waveform + dead-time on GPTW	**PASS**
`encoder_test/`	TPU quadrature counts and direction sign	**PASS**
`gpio_test/`	100+ GPIO toggles via external AD2 fixture	**PASS**
`uart_test/`	SCI loopback against debug USB-UART bridge	**PASS**
`imu_test/`	BNO055 orientation output over I2C	**PASS**
`sonar_test/`	HC-SR04 trigger/echo and range computation	**PASS**
`usb_test/`	USB CDC enumeration and byte echo	**PASS**

4 Gateway Tests (Go)

4.1 Scope and Inventory

Forty-six `*_test.go` files across twenty-one packages exercise the gateway's protocol stack and service layer. Key package-level test groups are shown in Table 4.

Table 4: Gateway test packages.

Package	Focus
`internal/frame`	Framing encode/decode, stream decoder, fuzz
`internal/fec`	Rate-1/2 K=7 convolutional encode/Viterbi decode
`internal/harq`	Chase Combining, mock transport integration
`internal/dispatcher`	Message routing, fuzzing
`internal/manager`	Session, heartbeat, health, hot-plug detection
`internal/link`	CDC edge cases, SPI framing
`internal/transport`	CDC and SPI mocks
`internal/controller`	Drive control, handler lifecycle
`internal/service`	gRPC services: motor, telemetry, firmware, configuration, gateway control
`internal/server`	gRPC, HTTP, config wiring
`internal/ws`	WebSocket hub and handler
`cmd/virtual_rx72n`	In-process RX72N simulator
`test/e2e`	End-to-end HIL test harness

4.2 Results and Coverage

Gateway tests run under `go test -race -coverprofile=coverage.out -covermode=atomic ./...`. The race detector is always on. Coverage is enforced at `MIN_COVERAGE=65%` by `.github/workflows/gateway.yml`.

Table 5: Gateway coverage summary from committed `star-gateway/coverage.out`.

Metric	Value
Packages with tests	21
Total test files	46
Total statement coverage	74.0%
CI gate	65.0%
Status vs. gate	**PASS** (+9.0 points above gate)
Race detector	enabled

Sub-package coverage spot-checks: `internal/fec` and `internal/frame/decoder.go` measure at 100.0 percent; the `ws/hub` hot-loop paths (`shouldThrottle`, `stampAndFanOut`) are at 100.0 percent and 77.8 percent respectively; the lowest-coverage routine flagged is `ws/hub.route` at 50.0 percent (pending a refactor to expose its private branches to unit tests).

4.3 Fuzz Testing

Go-native fuzz targets are committed in `internal/frame/fuzz_test.go` and `internal/dispatcher/fuzz_test.g`. They are run opportunistically by developers on branch builds and do not gate CI.

5 ROS2 Tests

5.1 Scope

Fifteen test files split between C++ (ament_cmake_gtest) and Python (ament_python with `launch_test`). Tests are invoked by `colcon test -return-code-on-test-failure` in `.github/workflows/ros2`

Table 6: ROS2 test inventory.

Package / Test	Verifies
star_spi_bridge / test_spi_driver.cpp	SPI driver unit behavior
star_spi_bridge / test_spi_message_converter	Protobuf round-trip
star_gateway_bridge / test_message_converter	Gateway bridge message codec
star_safety_monitor / test_safety_monitor	Heartbeat, stall, obstacle logic
star_perception / test_hailo_runner.py	Hailo inference runner shim
star_perception / test_depth_fusion.py	Stereo + Hailo fusion
star_bringup / launch_test_stereo_topics.py	Expected topics publish after launch
star_simulation / test_teleop_timeout.py	Teleop command timeout safety
star_simulation / test_ekf_drift.py	EKF drift bounds under simulated noise
star_simulation / test_stall_detection.py	Motor-stall triggers E-stop
star_simulation / test_slam_stability.py	SLAM convergence on synthetic scan
star_simulation / test_heartbeat_loss.py	Heartbeat-miss -> E-stop
star_simulation / test_nav2_recovery.py	Nav2 recovery behavior tree
star_simulation / test_obstacle_estop_latency.py	Obstacle -> E-stop latency budget

5.2 Results

All ROS2 tests pass on CI. The CI workflow uploads full test-result artifacts from `star-ros2/build/**/test_resul` on failure, and applies `uncrustify` C++ formatting as a separate gate.

5.3 Safety-Critical Tests

The `star_simulation` package contains four safety-critical behavior tests. Each is a `launch_test` that boots the required ROS2 nodes in-process, injects the stimulus, and asserts the expected response within a latency budget.

6 Compliance Engine Tests

6.1 Scope

Twelve pytest modules in `final_capstone/compliance-engine/tests/` verify the ADA-check algorithms and ROS2 node wrappers.

Table 7: Compliance engine test modules.

Test module	Focus
test_plane_segmentation.py	RANSAC plane extractor (ramp slope core)
test_yolo_fusion.py	YOLO + LiDAR fusion glue
test_doorway_lidar_detector.py	Door detection from scan
test_obstacle_clusterer.py	Clustering of obstacle points
test_cane_zone_filter.py	Cane-sweep zone extraction
test_corridor_medial_axis.py	Corridor medial-axis path width
test_imu_jolt_detector.py	IMU-based trip-hazard detection
test_door_offset_calibration.py	Door offset calibration
test_floor_frame.py	Ground-plane frame estimation
test_jamb_plane_fitter.py	Door jamb plane fit
test_nodes_smoke.py	ROS2 node smoke tests
test_nodes_behavior.py	Behavior tests on canned scans

6.2 Results and Coverage

Pytest is invoked with `pytest -maxfail=1 -cov=star_compliance -cov-report=xml -cov-report=term -junitxml=pytest-report.xml`. All tests pass at HEAD; coverage XML is uploaded as a workflow artifact. The approval stage of the four-phase `compliance-engine-ci.yml` workflow blocks merge if any unit test fails or if the integration-phase smoke test does not complete.

7 Protocol Buffers Tests

7.1 Cross-Language Serialization Tests

Three parallel test harnesses validate that Go, TypeScript, and nanopb encode identical bytes for the same canonical fixtures.

Table 8: Protobuf serialization test suites.

Language	Location	Coverage
Go	`star-proto/tests/go/`	5 tests (telemetry, config, firmware update, motor control, serialization round-trip)
TypeScript	`star-proto/tests/typescript/`	3 tests (telemetry, serialization, motor control)
nanopb (C)	`star-proto/tests/nanopb/`	Unity-based C serialization

7.2 Lint and Breaking-Change Gates

The `.github/workflows/proto.yml` workflow runs, in order: `buf lint` (style), `buf format` (formatting), `buf build` (schema integrity), and `buf breaking` against main on pull requests. A breaking change blocks merge unless the PR updates firmware, gateway, and ROS2 in the same change.

7.3 Results

All three serialization harnesses pass at HEAD. Buf lint, format, and build are green; no open breaking-change violations.

8 User Interface Tests

8.1 Scope

The legacy React operator UI at `star-ui/` is the subject of a single Vitest suite at `star-ui/src/hooks/useGamepad`. It verifies the gamepad hook's axis mapping, dead-zone handling, and 50 Hz command cadence. Coverage configuration is present but not gated in CI, because the primary operator dashboard (Grafana and Lichtblick) replaced the React UI as the main entry point late in development.

8.2 Status

Table 9: UI test status.

Suite	Status
`useGamepad.test.ts`	**PASS**

The gap between this single suite and the seventeen-panel inventory of the original React UI is acknowledged explicitly: the UI is [STRETCH] as delivered, since the Grafana and Lichtblick dashboards became the primary surfaces the grader will exercise. Tests for those dashboards exist at the protocol layer (gateway and compliance-engine tests cover every endpoint they invoke).

9 Integration and Simulation Tests

9.1 Virtual RX72N

`star-gateway/cmd/virtual_rx72n/` is an in-process Go program that reimplements the RX72N SPI peripheral surface against the gateway's transport layer. It runs inside the gateway test binary, so the full protocol stack (framing, FEC, HARQ, gRPC) can be exercised end-to-end without any wire. The associated test files `main_test.go` and `control_frames_test.go` ensure the simulator itself behaves identically to the real firmware at the observable layer.

9.2 End-to-End HIL Harness

`star-gateway/test/e2e/hil_test.go` drives the real RX72N over SPI from a test harness. It is skipped in standard `go test` invocations and triggered only by `.github/workflows/firmware-hil-nightly.yml` on a dedicated HIL fixture. Results are published as GitHub Actions artifacts.

9.3 ROS2 Launch Tests

Four behaviorally critical flows are covered by `launch_test` harnesses:

- Teleop timeout on missing heartbeat (`test_teleop_timeout.py`)
- EKF divergence bounds (`test_ekf_drift.py`)
- Motor stall -> E-stop sequence (`test_stall_detection.py`)
- Obstacle -> E-stop end-to-end latency (`test_obstacle_estop_latency.py`)

9.4 Protocol Bridge Layer Sanity

The gateway test suite exercises the complete protocol stack in simulation: a gRPC call enters at `internal/service/`, flows through `internal/harq/`, `internal/fec/`, and `internal/frame/`, exits via `internal/transport/` into the virtual RX72N, is decoded back up the stack, and the response is validated against the original request. This is the closest thing STAR has to a bit-exact protocol regression harness.

10 CI/CD Workflows

10.1 Workflow Inventory

Fifteen GitHub Actions workflows live under `.github/workflows/`. Table 10 lists the enforcement-bearing workflows.

Table 10: Enforcement-bearing CI workflows.

Workflow	What it enforces
firmware-unit-tests.yml	Unity suite, lcov HTML, 100/100/100 coverage on libs/
gateway.yml	Seven jobs: proto gen, build+test (race), lint, security, integration, benchmark, summary; `MIN_COVERAGE=65`
proto.yml	`buf lint/format/build`, breaking-change detection against main; Go, TS, and nanopb test runs
ros2.yml	`colcon test`, uncrustify format, artifact upload on failure
compliance-engine-ci.yml	Four phases: lint, unit, integration, approval; pytest + coverage + junit
firmware-hil-nightly.yml	Hardware-in-the-loop, nightly
ascii-check.yml	7-bit ASCII on every source file
copyright-check.yml	Header presence / consistency
firmware-build-verify.yml	RX72N release build succeeds
stm32-firmware.yml	STM32 firmware build (out of scope here)
firmware-toolchain-image.yml	GNURX container image cache
stm32-toolchain-image.yml	STM32 container image cache
proto-gen.yml	Regenerated artifacts match committed files
since-version-check.yml	`@since` tags present on public APIs

10.2 Pre-Commit Hooks

`.pre-commit-config.yaml` installs buf v1.50.1 hooks for `buf-lint`, `buf-format`, and `buf-generate` against `star-proto/proto`. The repository additionally uses a local hook at `scripts/git/pre-commit` to enforce the 7-bit ASCII policy; the same check is duplicated in `ascii-check.yml` so local hook bypass does not defeat CI.

10.3 Commit-Gate Summary

A pull request cannot merge to main unless: firmware coverage holds at 100 percent on `libs/`; gateway coverage is at least 65 percent; all ROS2 colcon tests pass; all compliance-engine pytest modules pass; buf lint and breaking-change checks are clean; 7-bit ASCII is satisfied; copyright headers are present; and generated protobuf artifacts match the committed files.

11 Consolidated Test Results

Table 11 is the top-level scorecard of every test category delivered on main at the date of this report.

Table 11: Consolidated test scorecard.

Category	Count	Status	Notes
Firmware unit (Unity)	46/46	**PASS**	100% coverage on libs/
Firmware bring-up (bench)	8 dirs	**PASS**	All on-bench
Gateway unit (Go)	46 files / 21 pkgs	**PASS**	74% coverage, 65% gate
Gateway fuzz (Go)	2 targets	advisory	Not gated in CI
ROS2 colcon	15 tests	**PASS**	All packages
ROS2 safety behavior	4 launch tests	**PASS**	E-stop, stall, EKF, teleop
Compliance pytest	12 modules	**PASS**	Coverage XML uploaded
Protobuf Go	5 tests	**PASS**	Serialization round-trip
Protobuf TS	3 tests	**PASS**	Serialization round-trip
Protobuf nanopb (C)	1 harness	**PASS**	Unity-based
UI Vitest	1 suite	**PASS**	Gamepad hook only
Integration (virtual_rx72n)	2 test files	**PASS**	In-process simulator
HIL nightly	1 harness	**PASS**	GitHub Actions artifact
Pre-commit ASCII	enforced	**PASS**	Local + CI
CI gates	14 workflows	**PASS**	On HEAD

The single visible shortfall is the sparseness of the React UI test suite. Because the operator surface migrated to Grafana and Lichtblick mid-semester, the React UI's test coverage was not expanded to match its original seventeen-panel scope; instead, the new dashboards rely on the gateway and compliance-engine test suites to validate every endpoint they invoke. This is recorded in Section 8.

12 Known Defects, Gaps, and Mitigations

12.1 Open Gaps

- **UI coverage** – single Vitest suite against a seventeen-panel inventory. Mitigation: the Grafana/Lichtblick dashboards are exercised at the API layer by gateway tests; the React UI is [STRETCH] rather than required.
- **`ws/hub.route` at 50 percent** – a private-branch coverage hole. Mitigation: a refactor is scheduled for post-capstone work; gateway total coverage still exceeds the 65 percent gate with headroom.
- **Fuzz targets advisory only** – Go fuzzing is not wired into CI time budgets. Mitigation: developers run fuzz targets opportunistically on frame and dispatcher changes.
- **Online PID autotuning** – not tested because not implemented (see SDD, Section on Known Limitations). PID gains are produced offline in MATLAB and verified via the `rx_pid` unit test against a reference trajectory.

12.2 Closed-Loop HIL Validation

The obstacle-to-E-stop latency test (`test_obstacle_estop_latency.py`) is the most safety-critical test in the suite. It verifies that a synthetic obstacle appearing inside the 10 cm `obstacle_estop_distance` threshold triggers a `/star/estop = true` publication inside one diagnostic period (100 ms), and that the gated velocity output returns to zero immediately thereafter. The test is green at HEAD.

12.3 Regression Policy

Breaking a CI gate requires the responsible PR to be reverted or the gate itself to be explicitly relaxed in the same PR. The project has no "skip CI" escape hatch; both branch protection and required status checks are enabled on main.

13 Conclusions

The STAR software delivers a broad, well-instrumented test suite: 46 firmware unit tests at 100 percent library coverage, 46 Go test files at 74 percent gateway coverage, fifteen ROS2 tests including four safety-critical launch tests, twelve compliance-engine pytest modules, nine Protobuf serialization tests spanning three languages, eight on-bench hardware bring-up programs, and fourteen CI workflows that enforce the gates described in Section 10. Every merge to main passes the full suite.

At the date of this report, the only visible gap is the React UI suite, which is acknowledged as [STRETCH] because the dashboarding surface migrated to Grafana and Lichtblick. Every other subsystem is at or above its intended coverage bar.

Appendix A: Reproducing the Test Runs

```
# Firmware unit tests
cd star-rx72n-firmware/tests
cmake -B build && cmake --build build
ctest --test-dir build --output-on-failure

# Gateway tests
cd star-gateway
go test -race -coverprofile=coverage.out -covermode=atomic ./...
go tool cover -func=coverage.out | tail -n 1   # total %

# ROS2 tests
cd star-ros2
colcon build --symlink-install
colcon test --return-code-on-test-failure --event-handlers console_direct
   +
colcon test-result --all

# Compliance engine
cd final_capstone/compliance-engine
pytest --cov=star_compliance --cov-report=term tests/

# Protobuf
cd star-proto
buf lint && buf format --diff && buf build
( cd tests/go         && go test -v ./... )
( cd tests/typescript && npm test )

# UI
cd star-ui
npm install && npm test
```

Appendix B: Key Test Artifacts

- `e2-studio-star-rx72n-firmware/tests/build/test-results.xml` – firmware JUnit XML, 46/46 tests, 0 failures.
- `star-gateway/coverage.out`, `star-gateway/coverage.html`, `star-gateway/coverage_transport.out` – gateway coverage outputs.
- `.github/workflows/*.yml` – all CI workflow definitions.
- `.pre-commit-config.yaml` – committed pre-commit hooks.
- `scripts/git/pre-commit` – ASCII enforcement hook.